Rita’s Pizza
Stephen Cox, Hayden Domino, Chandler Duffey, Harrison Crabb, Davos DeHoyos
Project Step 1

Professor Note: Please review dal-service project in github, and follow similar path to implement the interaction through api layer. Supabase for a maybe implementation

Requirements Gathering

Overview

Rita's Pizza is a multi-branch pizza franchise that needs a centralized database system to manage its day-to-day operations across all locations. Each branch operates semi-independently — it has its own staff, inventory, and order flow — but the franchise as a whole needs consistent tracking of employees, menu offerings, customer loyalty, and health inspections. The goal of this system is to provide a unified data model that supports operational reporting (e.g., branch sales, ingredient usage) while reflecting the real-world constraints of a food-service business.

Domain Research and Requirements

To gather requirements beyond the initial project brief, we examined how real pizza franchises operate. Branches typically maintain their own ingredient stock, reorder independently, and staff their own shifts. Customers may order in person at the counter (walk-ins) or online, and not every customer registers an account — walk-in orders are common and must be supported without requiring a customer record. Franchise loyalty programs reward repeat customers with redeemable offers, but each reward can only be used once. Health and safety inspections are conducted per location and must be logged with dates and results for compliance purposes.

Key Functional Requirements

The database must support the following:

- **Employee management:** Track employees by branch, role (General Manager, Shift Manager, Cook/Kitchen, Cashier/Front), name, hire date, and wage. Each branch must have exactly one General Manager.
- **Branch operations:** Store branch details including address, phone, operating hours, and seating capacity. Each branch manages its own inventory of ingredients with per-branch quantities and timestamps.
- **Order processing:** Record orders with date, type (online or in-person), and total price. Orders are always processed by a specific branch. An order may optionally be linked to a registered customer, or placed anonymously by a walk-in.
- **Menu and recipes:** Maintain a catalog of menu items with prices and categories. Track which ingredients each item requires and in what amounts (recipes).

- **Customer rewards:** Registered customers accumulate reward points and may earn redeemable rewards. Each reward can be redeemed at most once, on a single order. Walk-in customers cannot redeem rewards.
- **Health inspections:** Log inspection events per branch with date, result, and optional notes.

Business Rules

Branch
- **BR1.** Each branch must have exactly one General Manager (GMGR) at all times.
- **BR2.** A branch can have many employees, but each employee is assigned to at most one branch.
- **BR3.** Every employee must be assigned to a branch — no unassigned employees exist in the system.
- **BR4.** A branch can have many inspections over time; each inspection belongs to exactly one branch.

Employee
- **BR5.** Employee type is restricted to: GMGR, Shift Manager, Cook/Kitchen, Cashier/Front.
- **BR6.** Only an employee with type GMGR may manage a branch. A GMGR manages exactly one branch.

Order
- **BR7.** An order may be placed by a registered customer or anonymously (walk-in). Customer participation in the Places relationship is partial.
- **BR8.** Every order must be processed by exactly one branch.
- **BR9.** An order must contain at least one menu item.
- **BR10.** Order type is either 'online' or 'in-person'.

Rewards
- **BR11.** A reward is owned by exactly one customer; a customer may own many rewards.
- **BR12.** A reward may be redeemed at most once, linking it to one order. An order may redeem at most one reward.
- **BR13.** Only a registered customer can own or redeem a reward. Walk-in orders cannot redeem rewards.

Menu & Inventory
- **BR14.** A menu item uses one or more ingredients; an ingredient may appear in many menu items. The amount required is specific to each menu-item–ingredient pairing.
- **BR15.** Each branch maintains its own inventory. Inventory quantity and last-updated timestamp are specific to each branch–ingredient pair.

Summary and reasoning for each major relation

1. WorksAt between Employee and Branch

Cardinality: Employee N : 1 Branch

Reasoning:
The project says staff members work at a single location and employees are associated with at most one branch. That means many employees can belong to one branch, but each employee connects to only one branch.

Why this matters:

1. It models home branch assignment
2. It avoids putting employees at multiple branches unless your project later changes that rule

2. Manages between Employee and Branch

Cardinality: Employee 1 : 1 Branch, with participation limited to general managers

Reasoning:
The project explicitly says each branch must have one and only one general manager. So this is separate from WorksAt because not every employee manages a branch. Only employees with type GMGR should participate in this relationship.

Why separate it from WorksAt:

1. An employee can work at a branch without being its general manager
2. The rule “exactly one general manager per branch” is important enough to show as its own relationship

3. Places between Customer and Order

Cardinality: Customer 1 : N Order

Reasoning:
A customer can place many orders over time, but each order is placed by one customer. Since the project mentions customer rewards and online/in-person ordering, customer participation clearly matters in the business process.

Why include it:

1. Supports reward tracking
2. Supports customer order history
3. Connects customers to sales behavior

If our instructor allows walk-in orders with no customer account, we could mark customer participation in Places as partial.

4. ProcessedBy between Order and Branch

Cardinality: Branch 1 : N Order

Reasoning:
Orders happen at a specific Rita's Pizza location, and each branch operates independently with its own system. So one branch processes many orders, but each order belongs to exactly one branch.

Why this matters:

1. Lets you ask "which location performed the best?"
2. Ties orders and sales data back to a physical location

5. Contains between Order and MenuItem

Cardinality: M : N

Relationship attributes: quantity, maybe item_price

Reasoning:
One order can contain many menu items, and one menu item can appear in many different orders. That is a classic many-to-many relationship. In Chen notation, the relationship diamond is the right place for order-line details like quantity.

Why quantity belongs on the relationship:

1. Quantity is not a property of Order alone
2. Quantity is not a property of MenuItem alone
3. It depends on a specific pairing of one order and one menu item

This is the conceptual ER version of what later becomes an associative table like Order_Item.

6. Uses between MenuItem and Ingredient

Cardinality: M : N

Relationship attribute: amount_required

Reasoning:
A menu item uses multiple ingredients, and one ingredient can be used in many menu items. Since the database must track ingredients, menu, and food usage, this relationship is essential.

Why it is important:

1. Lets you connect sales to ingredient consumption
2. Supports queries like “How much dough has location X gone through?”
3. Gives the conceptual basis for recipes

Again, amount_required belongs on the relationship because it depends on the menu-item/ingredient pairing.

7. Stocks between Branch and Ingredient

Cardinality: M : N

Relationship attributes: quantity_on_hand, last_updated

Reasoning:
Each branch carries many ingredients, and the same ingredient can be stocked at many branches. Because each location maintains its own system and inventory, inventory is really about the relationship between a branch and an ingredient.

Why this is better than making Inventory a standalone entity in the conceptual ERD:

1. Conceptually, inventory describes how much of a given ingredient a branch has
2. That is naturally a relationship, not a separate real-world object at this stage

Later in the relational model, this often becomes an Inventory table.

8. Owns between Customer and Reward

Cardinality: Customer 1 : N Reward

Reasoning:

A customer can earn multiple rewards over time, but each reward belongs to one customer. The project specifically mentions customer rewards and asks which restaurant utilizes rewards the most.

Why include it:

1. Supports reward history
2. Connects customers to loyalty data
3. Makes reward usage measurable

9. Redeems between Reward and Order

Cardinality: Reward 0..1 : 1 Order

Reasoning:
The project says a reward can only be used once. That means one reward may be linked to at most one order. An order may redeem one reward.

Why this is its own relationship:

1. “Owning” a reward is different from “using” a reward
2. It helps model the one-time-use rule clearly

10. HasInspection between Branch and Inspection

Cardinality: Branch 1 : N Inspection

Reasoning:
The system must track safety information such as inspection logs and refrigerator temperatures. A branch can have many inspections over time, but each inspection belongs to one branch.

Why this matters:

1. Supports safety compliance
2. Lets each branch maintain a history of inspections
3. Fits the project requirement without overcomplicating the diagram

Design Decisions

1. WorksAt vs Manages as Separate Relationships

We considered combining both into a single relationship with a role attribute, but kept them separate because the participation rules differ: all employees participate in WorksAt (total participation), but only GMGRs participate in Manages (partial, restricted by type). Keeping them separate makes the constraint explicit in the ER diagram and avoids ambiguity.

2. Walk-in Orders (Partial Participation in Places)

We considered requiring all orders to have a customer account (total participation). We chose partial participation because Rita's Pizza serves walk-in customers who may not want to register, which is realistic for a pizza counter. Rewards cannot be applied to walk-in orders (BR13), so the system remains consistent.

3. Inventory as a Relationship, Not an Entity

At the conceptual ER level, inventory describes how much of a given ingredient a branch currently has — this is naturally a property of the Branch–Ingredient relationship (Stocks), not a standalone real-world object. At the relational level it becomes an INVENTORY table with a composite key, but the conceptual design intent is relationship-based.

4. Employee Type as an Attribute, Not a Subtype (ISA)

We considered making GMGR, Shift Manager, Cook/Kitchen, and Cashier/Front into subtypes using an ISA hierarchy. We chose a single `type` attribute instead because the subtypes share all the same attributes — there is no attribute unique to one type. The only behavioral difference is the Manages relationship, which is captured separately.

5. Attribute Reconciliation and Naming Conventions

During the design process, team members independently proposed slightly different attribute sets. We reconciled these as follows:
- **Branch:** We used specific address fields (`street_addr`, `city`) rather than a single `location` field, for better queryability. We dropped `reputation` as it is subjective and not captured by a transactional system.

- **Employee:** We merged `role` and `employee_type` into a single `type` attribute to avoid redundancy. We added `wage` from the teammate's proposal as it is operationally useful.
- **Order:** We standardized on `total_price` over `total_amount` for consistency with MenuItem's `price` field.
- **Ingredient:** We added `cost_per_unit` (useful for inventory valuation) but dropped `expiration_date` (which belongs at the inventory/batch level, not the ingredient definition).
- **Reward:** We added `reward_type`, `issue_date`, and `used_status` to support the one-time redemption rule (BR12).
- **Inspection:** We kept both `result` (structured outcome) and `notes` (free-text details).

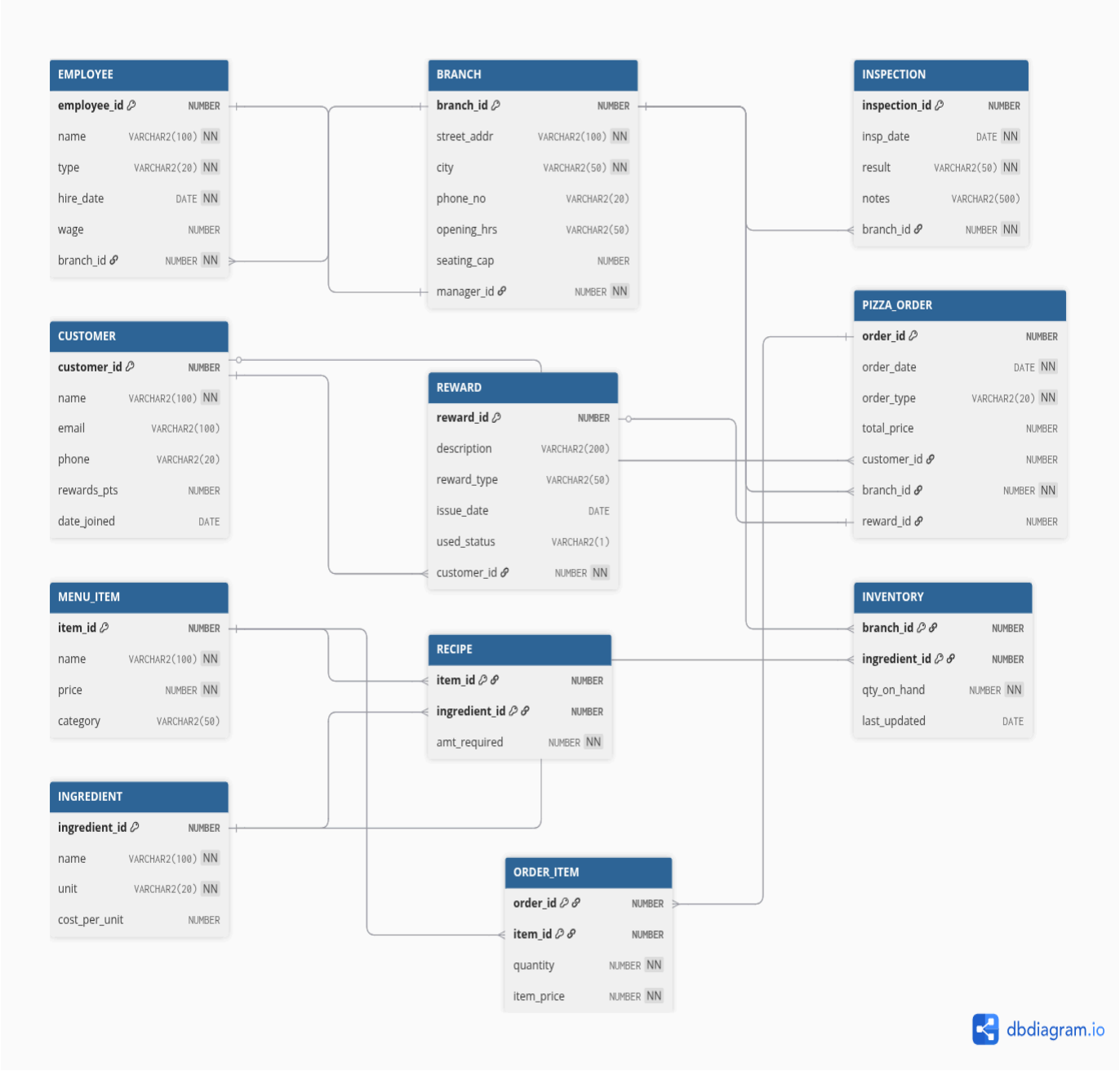
6. Redeems as a Separate Relationship

We modeled reward redemption as its own relationship (Redeems) between Reward and Order, rather than just a foreign key on Order. This makes the one-time-use constraint (BR12) explicit: a reward participates in Redeems at most once (0..1 cardinality on both sides). It also cleanly separates ownership (Owns) from usage (Redeems).

Assumptions

- A branch always has exactly one GMGR; the system does not track historical manager assignments.
- Menu items are franchise-wide (not branch-specific). All branches offer the same menu.
- Ingredient definitions are global; only inventory quantities vary by branch.
- An order belongs to exactly one branch and cannot span multiple locations.
- Reward points are tracked as a running balance on the Customer entity; individual point transactions are not modeled.

Relational Schema



Relational Schema

Primary keys are **bolded**. Foreign keys are marked with → referencing their parent table.

BRANCH
Column	Type	Constraints
branch_id	NUMBER	PK
street_addr	VARCHAR2(100)	NOT NULL
city	VARCHAR2(50)	NOT NULL
phone_no	VARCHAR2(20)	
opening_hrs	VARCHAR2(50)	
seating_cap	NUMBER	
manager_id	NUMBER	FK → EMPLOYEE(employee_id), UNIQUE, NOT NULL

EMPLOYEE
Column	Type	Constraints
employee_id	NUMBER	PK
name	VARCHAR2(100)	NOT NULL
type	VARCHAR2(20)	NOT NULL, CHECK IN ('GMGR','Shift Manager','Cook/Kitchen','Cashier/Front')
hire_date	DATE	NOT NULL
wage	NUMBER(8,2)	
branch_id	NUMBER	FK → BRANCH(branch_id), NOT NULL

CUSTOMER
Column	Type	Constraints
customer_id	NUMBER	PK
name	VARCHAR2(100)	NOT NULL
email	VARCHAR2(100)	
phone	VARCHAR2(20)	
rewards_pts	NUMBER	DEFAULT 0
date_joined	DATE	

PIZZA_ORDER

Column	Type	Constraints
order_id	NUMBER	PK
order_date	DATE	NOT NULL
order_type	VARCHAR2(20)	NOT NULL, CHECK IN ('online','in-person')
total_price	NUMBER(10,2)	
customer_id	NUMBER	FK → CUSTOMER(customer_id), NULLABLE
branch_id	NUMBER	FK → BRANCH(branch_id), NOT NULL

> Table named PIZZA_ORDER instead of ORDER because ORDER is a reserved word in Oracle SQL.

MENU_ITEM

Column	Type	Constraints
item_id	NUMBER	PK
name	VARCHAR2(100)	NOT NULL
price	NUMBER(8,2)	NOT NULL
category	VARCHAR2(50)	

INGREDIENT

Column	Type	Constraints
ingredient_id	NUMBER	PK
name	VARCHAR2(100)	NOT NULL
unit	VARCHAR2(20)	NOT NULL
cost_per_unit	NUMBER(8,2)	

REWARD

Column	Type	Constraints
reward_id	NUMBER	PK
description	VARCHAR2(200)	
reward_type	VARCHAR2(50)	

issue_date	DATE	
used_status	VARCHAR2(1)	DEFAULT 'N', CHECK IN ('Y','N')
customer_id	NUMBER	FK → CUSTOMER(customer_id), NOT NULL

INSPECTION
Column	Type	Constraints
inspection_id	NUMBER	PK
insp_date	DATE	NOT NULL
result	VARCHAR2(50)	NOT NULL
notes	VARCHAR2(500)	
branch_id	NUMBER	FK → BRANCH(branch_id), NOT NULL

ORDER_ITEM (associative — resolves Contains M:N)
Column	Type	Constraints
order_id	NUMBER	PK, FK → PIZZA_ORDER(order_id)
item_id	NUMBER	PK, FK → MENU_ITEM(item_id)
quantity	NUMBER	NOT NULL
item_price	NUMBER(8,2)	NOT NULL

> Composite PK: (order_id, item_id)

RECIPE (associative — resolves Uses M:N)
Column	Type	Constraints
item_id	NUMBER	PK, FK → MENU_ITEM(item_id)
ingredient_id	NUMBER	PK, FK → INGREDIENT(ingredient_id)
amt_required	NUMBER(8,2)	NOT NULL

> Composite PK: (item_id, ingredient_id)

INVENTORY (associative — resolves Stocks M:N)
Column	Type	Constraints

branch_id	NUMBER	PK, FK → BRANCH(branch_id)
ingredient_id	NUMBER	PK, FK → INGREDIENT(ingredient_id)
qty_on_hand	NUMBER	NOT NULL
last_updated	DATE	

> Composite PK: (branch_id, ingredient_id)

Relationship: Redeems (Reward ↔ Order)

The Redeems relationship (0..1 : 0..1) is implemented via a foreign key on PIZZA_ORDER:

Column	Type	Constraints
reward_id	NUMBER	FK → REWARD(reward_id), NULLABLE, UNIQUE

> Added to the PIZZA_ORDER table. UNIQUE ensures a reward is redeemed at most once. NULLABLE allows orders without a reward.

—

Constraints Not Expressible in Relational Schema

The following business rules cannot be fully enforced through table constraints alone and would require triggers or application logic:

1. **BR1** — "Each branch must have exactly one GMGR at all times." The UNIQUE FK `manager_id` on BRANCH ensures at most one, but enforcing that the referenced employee has `type = 'GMGR'` requires a trigger or CHECK constraint with a subquery (not supported in Oracle CHECK).
2. **BR6** — "Only a GMGR may manage a branch." Same issue — cross-table validation needed.
3. **BR9** — "An order must contain at least one menu item." Minimum participation in ORDER_ITEM cannot be enforced declaratively; requires a trigger or application-level check.
4. **BR13** — "Only registered customers can redeem rewards." Ensuring that if `reward_id` is set on PIZZA_ORDER then `customer_id` is also NOT NULL requires a trigger or compound CHECK.
5. **Circular FK between BRANCH and EMPLOYEE** — BRANCH references EMPLOYEE (manager_id) and EMPLOYEE references BRANCH (branch_id). This creates a chicken-and-egg insertion problem that must be handled with deferred constraints or by inserting with NULLs first and updating afterward.

Chen Diagram:

```
                              (location)
                                  |
                            (_branch_id_)
                                  |
                             (reputation)
                                  |
                              [BRANCH]
                              /   |   \
                         1..N/    |1   \1..N
                            /     |     \
                           /   <Manages>  <HasInspection>
                          /       |         \
                     <WorksAt>    |          \       (_inspection_id_)
                        /         |           \            |
                  N..1/          1..1          \         (inspection_date)
                     /                           \       (result)
               [EMPLOYEE]                         [INSPECTION]
               /   |   |  \                            |
      (_employee_id_) |   |   \                      (notes)
             |        |   |    \
          (name)   (wage) |   (certification_status)
                          |
                   (employee_type)
                          |
                       (role)


 [CUSTOMER] 1..N <Places> 1..1 [ORDER] N..1 <ProcessedBy> 1..N [BRANCH]
    |                              /   |   \
(_customer_id_)          (_order_id_) |  (total_amount)
    |                         |        |
  (name)                 (order_date) (order_type)
    |
 (phone)


 [ORDER] M..N <Contains> M..N [MENU_ITEM]
                  |
               (quantity)
                  |
              (item_price)

 [MENU_ITEM] M..N <Uses> M..N [INGREDIENT]
     |                  |
(_menu_item_id_)     (_ingredient_id_)
     |                  |
  (item_name)        (ingredient_name)
     |                  |
   (price)           (cost_per_unit)
     |                  |
 (category)          (expiration_date)

 [BRANCH] M..N <Stocks> M..N [INGREDIENT]
                  |
         (quantity_on_hand)
                  |
            (last_updated)

 [CUSTOMER] 1..N <Owns> 1..1 [REWARD] 0..1 <Redeems> 1..1 [ORDER]
                                  |
                            (_reward_id_)
                                  |
                            (reward_type)
                                  |
                             (issue_date)
                                  |
                             (used_status)
```

KEY:

Rectangle = entity []
Oval = attribute ()
Underlined oval = key (_something_)
Diamond = relationship <>
Cardinality goes on the connecting lines
1 .. N one to None, M .. M Many to Many ETC